

South-East Technical University



HDIP in Computer Science

Final Report

***APAVS: Automated pipeline and visualization
system for a Tool's Load Port Data***

Student Name: Vincent Byrne

Lecturer: Sonya Hogan

Student Number: 20108898

Date: 30/03/2026

Declaration of Authenticity

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student.......... Date 30/03/2026

Work Place Mentor.......... Date 31/3/26

The word count of the main body of text:7725

Acknowledgements

I would like to thank my project supervisor Sonya Hogan for her weekly guidance and feedback throughout the duration of the project. I would also like to thank Kevin M O'Dea, DM at Intel, for providing his support in providing insight to the tool environment and knowledge of the load port qualification process.

Abstract

APAVS automates the monitoring of load port qualification data across a fleet of semiconductor tools in one of Intel's manufacturing sites. Technicians and Engineers previously had to rely on manually exporting log files, running Excel macros, and inspecting tools post qualification without the possibility of have fleet-wide visibility. APAVS replaced the previous workflow with a PowerShell extraction pipeline that connects to tool hard drives via a network share, parsing raw sensor logs, and calculating the slot-level offsets against its taught reference position. Three tiers of classification were used (Pass, Fail, Incomplete) after it was noticed that 0.00 zero values were being incorrectly classified as failures. Results are imported to an SQL server database and visualised through a Power BI dashboard providing fleet-wide health report, trend analysis, and drill-down to slot specific measurements. The pipeline was tested against the CSB Fab24 tool set and was compared to previous Excel macro. Over 10,000 qualification runs were processed from historical data.

Key Words

Semiconductor manufacturing, load port qualification, data pipeline, PowerShell automation, SQL Server, Power BI, fleet monitoring, offset analysis

List of Abbreviations

APAVS Automated Pipeline and Visualisation System

BKM Best Known Method

CEID Common Equipment Identifier

CSV Comma-Separated Values

DAX Data Analysis Expressions

FOUP Front Opening Unified Pod

FI Factory Interface

LP Load Port

MES Manufacturing Execution System

MVP Minimum Viable Product

OOC Out of Control

PVD Physical Vapour Deposition

SMB Server Message Block

SSMS SQL Server Management Studio

UNC Universal Naming Convention

VBA Visual Basic for Applications

Table of Contents

1	Introduction	1
2	Research.....	2
2.1	Market Analysis.....	2
2.2	Technical Research.....	3
3	Analysis	4
3.1	Current State Workflow	4
3.2	User Personas	6
3.3	User Requirements	7
3.4	Feature Analysis	8
4	Design.....	10
4.1	System Architecture.....	10
4.2	Screen Design.....	11
4.3	Walkthrough	11
4.4	Data Model	12
5	Methodology.....	12
5.1	Development Approach (Methodology Type)	12
5.2	Technologies	12
6	Project Plan	13
6.1	Sprint Schedule	13
7	Implementation	14
7.1	Overview of Implementation.....	14
7.2	MVP Dashboard	14
7.3	Phase 2: Database Migration and Enhanced Visualisation	18
7.4	Phase 3: Slot-Level Measurement and Trend Analysis.....	20
7.5	Phase 4: Automated Extraction Pipeline	21
8	Future Work.....	26
8.1	Automated Scheduling.....	26

8.2	Global Database Environment	27
8.3	Historical TaughtZ Position	27
8.4	Drift Alerting	27
8.5	Expansion to Additional Tool Sets	27
8.6	Overview	28
9	Critical Self Review.....	29
9.1	What I have learned.....	29
9.2	What I Achieved.....	31
9.3	Problems Encountered	33
9.4	Use of Generative AI	35
10	References	36

List of Figures

Figure 1.1. Three Load Ports of an Endura PVD Tool (Bridge Tronic Global, 2025)	1
Figure 4.1. System Architecture Diagram	10
Figure 4.2. Mock Prototype of Screen Design of Fleet	11
Figure 6.1. Project Sprint Schedule	13
Figure 7.1. Original File Structure	14
Figure 7.2. Revised File Structure	15
Figure 7.3. Entity-Relationship Diagram for MVP	16
Figure 7.4. MVP Dashboard Layout	18
Figure 7.5. Conditional Formatting Included in the Overall Result Column	19
Figure 7.6. Updated ER Diagram	20
Figure 7.7. Trend Chart of a Tool	21
Figure 7.8. APAVS Data Pipeline from Tool Network to Development Laptop	23
Figure 7.9. Old and New Logic with and Without Incomplete Category	24
Figure 7.10. Mismatch of TaughtZ Value Over Time	25
Figure 7.11. Process for a Single Tool	26
Figure 7.12. Extraction Complete Message	26
Figure 9.1. Tool Detail View	31

List of Tables

Table 3.1. Technician Attributes and Description.....	6
Table 3.2. Engineer Attributes and Description.....	7
Table 3.3. MVP Features (Must have)	9
Table 3.4. Advanced Features (Future Goals).....	9
Table 5.1. Project Tools.....	13
Table 7.1. MappingData.log Field and Descriptions	15
Table 8.1. User Stories and Status	28

1 Introduction

Semiconductor manufacturing is one of the most precision dependent industries in the world. A process tool must operate within extremely tight tolerances to ensure an expected yield. One of the very many components of this industry is the qualification of load ports, which are the interfaces in which wafers are transported between FOUPs (Front Opening Unified Pods) and the tool robotic wafer handling system. An example of a load port station can be found below in Figure 1.1.



Figure 1.1. Three Load Ports of an Endura PVD Tool (Bridge Tronic Global, 2025)

This project addressed the inefficiencies of qualifying the load ports at one of Intel's manufacturing facilities. As a technician working on the CSB tool set in the metals area, the manual process to qualify the tools was labourousome and there was no way to review all the tools' load port data without many manual steps. The current Best-Known Method (BKM)

requires technicians to export the log files from the tool's software, retrieve robot configuration data, and transfer the files to a Tool Admin server. Once the files have been transferred an Excel Macro generates offset plots for the visual inspection. Technicians or Engineers can review these plots to determine whether they fall into pass or fail criteria. The goal was to automate this workflow and create a mechanism for monitoring trends across the tool fleet, alerting engineers to drift in wafer placement as it occurs.

2 Research

2.1 Market Analysis

Semiconductor manufacturing implements a vast range of software for equipment monitoring and process control. A specific software solution for addressing the problem outlined in the introduction is not yet available but a quick overview of the current existing commercial offerings will be briefly analysed.

Enterprise Manufacturing Execution Systems

The Semiconductor software market is dominated by Large-scale MES platforms. IBM has SiView which provides statistical process control for real-time monitoring of equipment data trends, supporting over 200 ready-to-use fab operation scenarios for full automation (IBM, 2024). Another MES solution is offered by Critical Manufacturing. The software monitors equipment uptime, idle time, and error states with the capability to view it in through real time their fabLive interface (Critical Manufacturing, 2026). These systems are integrated into the equipment factory wide and communicate on standards such as SECS/GEM and provide real time information that allow for material tracking and process control.

Yield Analytics Platforms

Software specialised in yield management focus on defect detection and optimising production. One of KLA's defect detection and data management systems *Klarity* helps fabs achieve faster yield learning through real-time excursion identification. The *Klarity* management system also allows manufacturers to take corrective action sooner allowing a better time to market (KLA, 2023). Another yield management tool is yieldWerx which provides real-time performance metrics which include advanced geospatial pattern recognition for identification of variation across production flows. (yieldWerx, 2026.)

Open-Source Projects

GitHub offers a variety of semiconductor management projects although none would directly address load port qualification. One open-source project which provides a dashboard for uploading production data is the Semiconductor-Insights repository, this repo allows for time series analysis and machine learning for yield prediction (Hosseinglm, 2025). Another is the SECOM Project which analyses sensor data for fault detection in wafer fabrication which will enable an increase in process throughput and unit production cost. (shirishpandagare, 2025). These projects focus more on yield prediction rather than tool qualification.

Identified Gap

The market analysis reveals a gap in the enterprise-level MES systems and the specific need of load port qualification monitoring. The previously mentioned SiView and Critical Manufacturing MES provide comprehensive factory automation but does not have tools to our specific use case. The other yield analytic tools address wafer defect detection but nothing specifically on load port qualification. For now, there is no existing solution, in the commercial world or open source, that addresses the specific needs of robot mapping for load port qualification and decision-making.

The current state at Intel reflects this gap as technicians and engineers rely on locally run Excel macros that generate qualification plots for specific load ports. There is also no real time information or fleet-wide monitoring capabilities. This project will address these missing capabilities by implementing a fleet board monitoring and more user-friendly analysis.

2.2 Technical Research

This Section will examine the technologies under consideration for implementation for the project.

Visualisation Platform: Power BI

Microsoft Power BI is selected for the visualisation platform for several reasons. First it is already deployed within Intel's infrastructure and utilised for daily use by technicians and Engineers which would reduce the barriers for adoption. It also allows for interactive filtering and conditional formatting which would highlight the pass/fail nature of load port

qualification. It allowed allow for direct connectivity to SQL databases with can refresh data on intervals allowing for real-time data visibility. It would also allow for a valuable professional development opportunity aligned with Intel's needs.

Database: SQL Server

An SQL database was used to store the qualification data, which will enable historical analysis and trend detection which can be used to send alerts. The SQL server was also used for its query capabilities needed for aggregating data across multiple tools, calculating trends, and identifying statistical outliers. The structured nature of the tool qualification data with entities such as tools, load ports, run times, and slot measurements translated well to a relational model.

Considerations for Data Pipeline

The initial phase of the project focused on analysis and visualisation using manually prepared synthetic data. Later iterations automated data extraction from tool logs using PowerShell. In the current workflow two data sources, FI logs containing Z-position mapping and data for each slot and Robot Config files containing taught positions against which the offsets and then calculated. These files resided on tool hard drives and were retrieved programmatically in the final phase of the project.

3 Analysis

3.1 Current State Workflow

Firstly, an understanding of the existing process is needed to design an effective solution. The current Best-Known Method for load port qualification involves the current workflow:

Step 1: Data export from Tool

Following a load port repair, the FI logs are retrieved from the tool's Millenium software. This requires navigating to **System** -> **Maintenance** and executing the "Get Fic Logs" Function. These logs are saved to a default location that varies by software version. (c:\ficlogs\ or d:\export\cgafi\).

Step 2: Robot Configuration File Export

The technician must then export the Robot1 configuration file which contains the taught positions. This requires idling the tool and putting the robot in local mode. The configuration is then exported to C:/ on the tool.

Step 3: Remote Macro Execution

The technician connects to a fab-specific Tool Admin service using Remote Desktop and downloads the appropriate FI Mapping Macro Excel file. The macro is configured with the target tool name “CSB801” for example, and it remotely retrieves the latest logs and configuration from the hard drive.

Step 4: Data Analysis

The macro will then extract the mapping data and calculate the offsets between the mapped and taught positions for each of the 25 slots. Plots will also be labelled by port number. The offset calculation is:

$$\text{Slot } N \text{ offset} = \text{Slot } N \text{ Mapped Position} - (\text{Taught position} + (10 \cdot (N - 1)))$$

Step 5: Qualification Decision

An experienced technician or engineer would then visually inspect the plots to verify all data points fall within the $\pm 1\text{mm}$ limits. For tools that were not recently repaired the Engineer would check for unexplained shifts and monitors if the mean distribution has shifted $< 0.4\text{mm}$ would which indicate an issue requiring further investigation.

Workflow Issues

The workflow as it stands has needed many different manual interventions across different systems. Analysis only really exists as an individual Excel session with no persistence. There is not yet a mechanism for historical tracking and requires manually rerunning to compare output across the fleet. There was also no alerting system for when robot centring was drifting outside the specifications.

3.2 User Personas

User personas will be created to help inform the modelling of the project. Understanding their needs will help inform the dashboard design with load port qualification data.

Persona 1: Technician (post-repair)

Attribute	Description
Role	Blue badge manufacturing technician in metal area.
Trigger	Completed a load port repair and logged 25 mech wafers for qualification.
Context	Typically handling other tasks or on break while mechs complete (under one hour).
Question	Did the repair work? Are all slot offsets within $\pm 1\text{mm}$ specification?
Decision	Pass: Bring tool back UTP if no other work pending. Fail: reteach positions and retest.
Time Pressure	Variable based on tool availability.

Table 3.1. Technician Attributes and Description

The technician requires a clear, unambiguous pass/fail indication. When load port qualification fails, the specific slots which are out of specification need identified so that a reteach can be completed. Current state accuracy is important.

Persona 2: Engineer (Fleet Monitoring)

Attribute	Description
Role	Engineer responsible for CSB tool fleet in metal area.
Trigger	Periodic review of tool health, not tied to specific repair events.
Context	Proactive maintenance planning and issue identification.
Question	Are any tools drifting toward specification limits? Which need attention?
Decision	Prioritise tools for preventive maintenance or investigation.
Time Pressure	Lower urgency but high value in early detection of emerging problems.

Table 3.2. Engineer Attributes and Description

The engineer needs to have visibility of the entirety of the fleet showing all tools at a glance with indicators of health. Trend data would be valuable for identifying drift before tool the values go out of control.

3.3 User Requirements

Now that we have our personas, we can create user stories which is a technique used in agile software development to capture product requirements from the perspective of the end user. User stories are used to articulate what the user wants and why to assist in learning about the product that is being developed. (Cohn, 2019). The structure of these user stories usually follows:

As a < type of user >, I want < some goal > so that < some reason >

Technician Stories

US-01: As a technician, I want to see the pass/fail status post repair of a tools load ports after the mechs have completed, so I can confirm the maintenance was successful.

US-02: As a technician, I want to see which slots are out of specification, so I know what positions need a retest.

US-03: As a technician, I want to see when the data was last updated, so I know I'm looking at information that is current and important.

Engineer Stories

US-04: As an Engineer, I want to see the health status across the CSB fleet at a glance, so that I can schedule maintenance activities if needed.

US-05: As an Engineer, I want to see trend data for all tools, so I can monitor for drifts before the load port goes OOC.

US-06: As an Engineer, I want to filter the view by tool or specific load port, so I can find the specific information quickly.

Shared Stories

US-07: As a user, I want to be able to view the dashboard and see a clear indication of the pass/fail criteria using colour coding, so I can quickly assess the status of the load ports.

US-08: As a user, I want to be able to have a fleet overview to individual tool/load port details, so I can investigate issues as appropriate.

3.4 Feature Analysis

MVP Features

Based on the user requirements we can now prioritise the elements for the Minimum Viable Product and future enhancements if there is time for completion. These elements were prioritised by the immediate needs of the technician and the engineer. For instance, the most essential information that is required by the technician is if the tool that has just undergone a maintenance activity passing its load port mapping. What information does the technician need to determine this passing state? The technician needs the specific identify of the tool and load port, the pass and fail calculation, the time stamp showing it was in a passing state after the maintenance activity took place. The engineer's immediate needs are much the same but would an overview of the entirety of the fleet to see which tools are in control. The MVP features can be seen in Table 3.3 on page 6.

Feature	Description	User Stories
Fleet Overview	Summary View showing all tools with pass/fail status.	US-04,US-07
Tool Detail View	Individual tool page showing load port status and slot specific data.	US-01,US-02
Pass/Fail Calculation	Automated comparison of offsets against $\pm 1\text{mm}$ specification.	US-01,US-07
Time Stamp	Timestamp showing when data was last updated.	US-03
Filtering	Ability to filter data by tool and load port.	US-06

Table 3.3. MVP Features (Must have)

Advanced Features

The more advanced features would be a stretch goal for the project. These features would be great additions to the project but would not be essential for the functionality of post maintenance qualification. Drift alerting is a feature not yet implemented with the current operation of load port maintenance, it would be an interesting stretch goal and upon tackling this feature it may become apparent was this feature hasn't yet already been implemented. The advanced features for the APAVS project can be seen in Table 3.4 below.

Feature	Description	User Stories
Historical Trending	Charts showing historical data over time for detection of drift.	US-05
Drift Alerting	Factory communications / alerts when mean offset exceeds threshold.	US-05
Tool Detail Navigation	Advanced navigation to see specific tool detail.	US-08
Export Capability	Ability to export data for sharing.	N/A

Table 3.4. Advanced Features (Future Goals)

4 Design

4.1 System Architecture

The system comprises of four individual layers: data input, data storage, data processing, and presentation. As can be seen below in Figure 4.1 the data was first to be input manually using AI to help generate anonymised data, then in future iterations of the project this was automated by extracting tool logs from the tools using PowerShell. The data processing layer used Power BI DAX to complete the necessary calculations where they were then presented on a dashboard for all users to view.

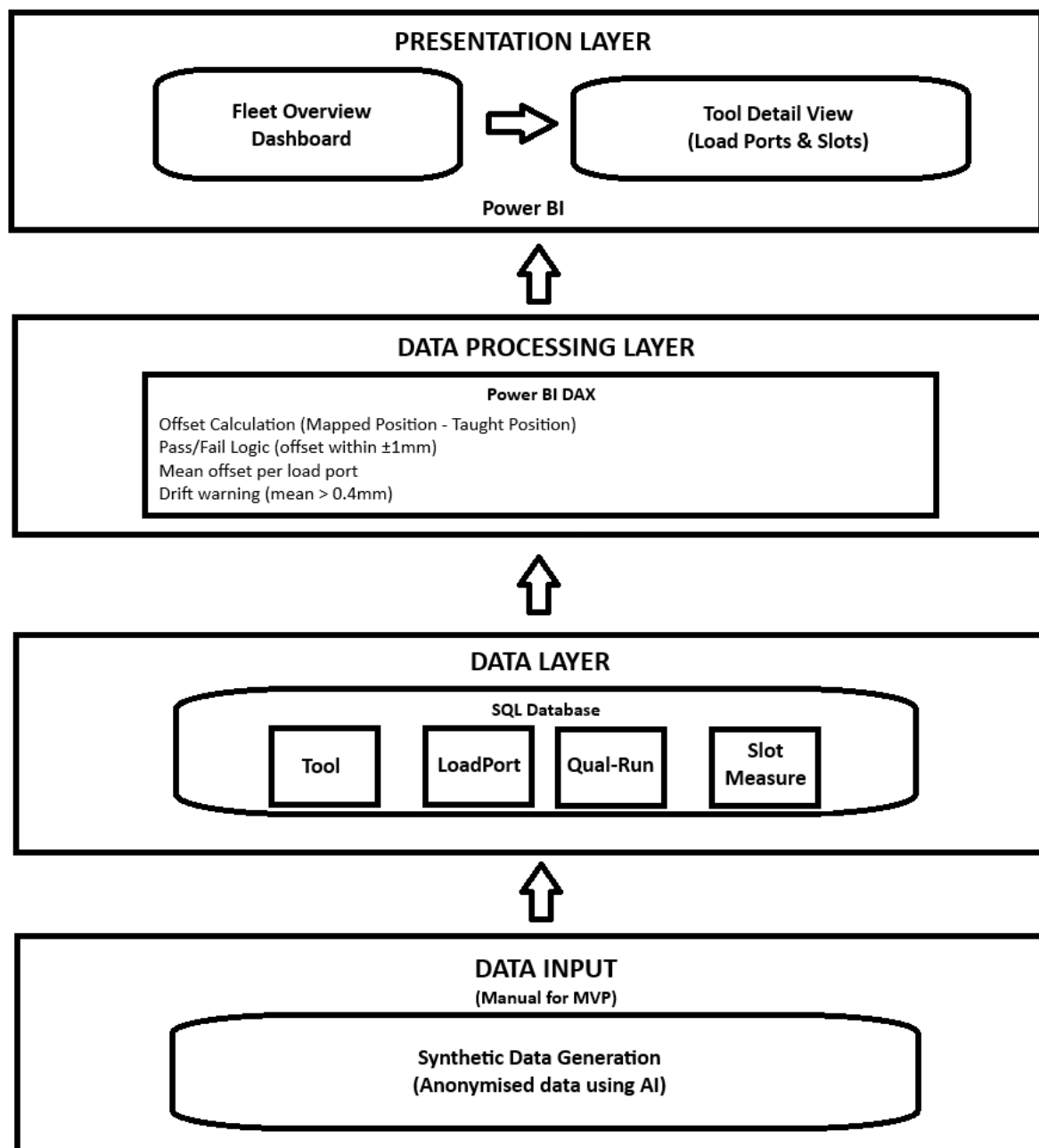


Figure 4.1. System Architecture Diagram

4.2 Screen Design

The screen design displayed the status of each individual tool and its associated load ports. The fundamental principle of the screen design was to provide a quick pass/fail status of each load saving time and not bogging the user down digging through information.

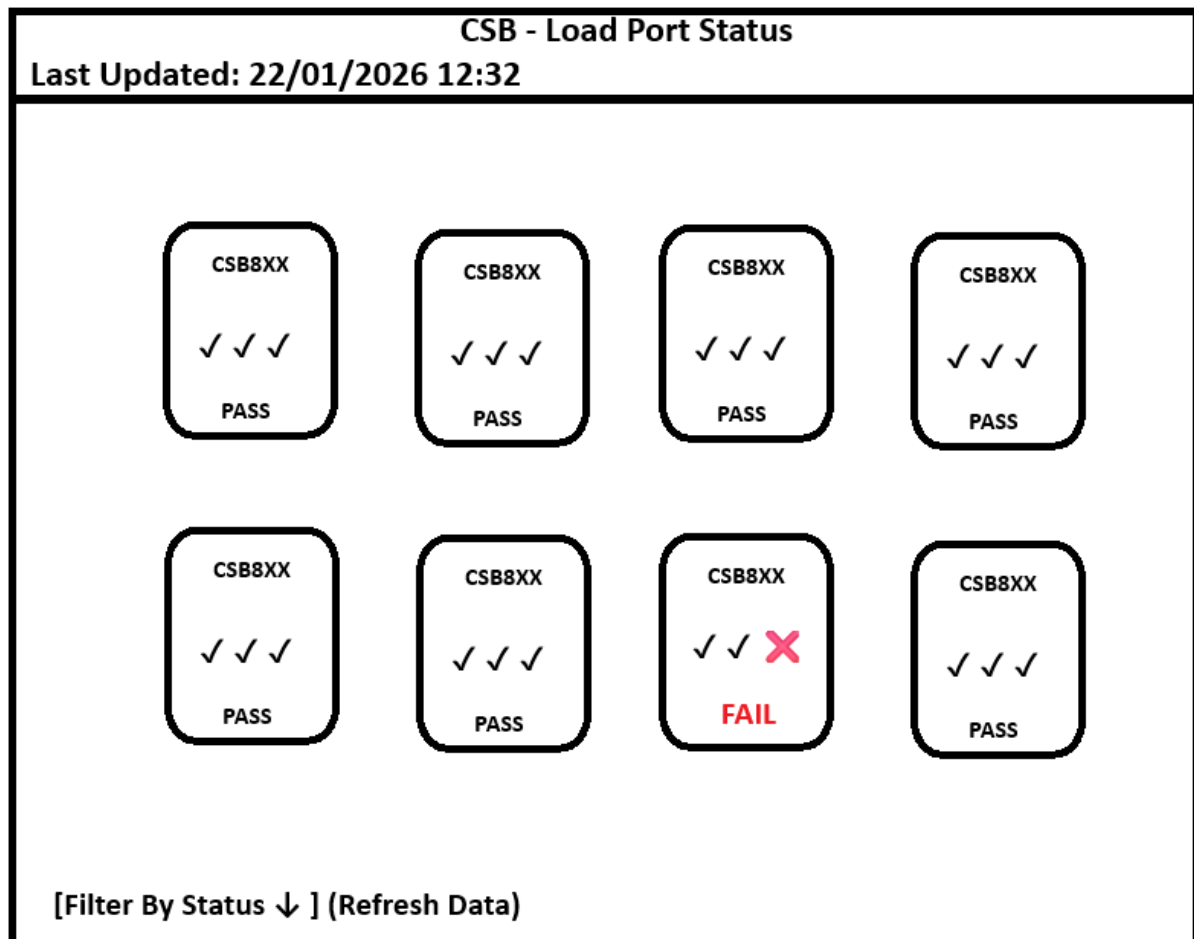


Figure 4.2. Mock Prototype of Screen Design of Fleet

4.3 Walkthrough

Scenario 1: Post-Repair Qualification Check

A technician has finished maintenance of CSBXXX. They will open the APAVS dashboard and select that specific tool from the tool filter. The dashboard will display the most recent qualification run showing a clear indication of Pass or Fail status. If it's a pass the technician will continue with full tool qualification. If it's a failure the specific load ports can be observed and will coordinate with the engineers for next steps.

Scenario 2: Monitoring Fleet health

An engineer will open up the dashboard for review when going through the morning passdown. They will observe the KPI cards showing the overall fleet health. If any tools are out of control they can filter by Fail and look at the tools with failures. With enhanced features a trend chart will be made available which will enable a notification for tools which are drifting towards the specification limits.

4.4 Data Model

Source Data Analysis

There are three files produced by the CSB tool software that need to be understood before we can begin to design the data base.

RobotConfig.log.txt contains the taught Z-position reference values for each load port. For the CSB tool set, both P1 and P2 ports share a taught Z value of 22.82mm.

MappingData.log contains the Z-position for all 25 slots.

FI_log files contain additional diagnostic information.

5 Methodology

5.1 Development Approach (Methodology Type)

This project followed the Agile methodology approach. This methodology was well suited to the project as it followed the one- or two-week sprints recommended by the handbook and made weekly meetings with the project supervisor more productive. Requirements were reevaluated as needed, and the solution was tested with incremental deliveries which allowed for earlier feedback.

5.2 Technologies

The technology stack for this project is a combination of tools for project management, visualisation, and data storage. Also included are a few applications which allowed for file sharing and communication to the supervisor and intel engineers. The technologies used can be seen below in Table 5.1.

Tool	Purpose
Trello	Sprint planning, progress tracking
GitHub	Upload versions of project, show project timeline
Power BI Desktop	Dashboard development and testing
SQL	Database for loapport and relevent data
Microsoft Word	Report writing and documentation
Slack	Communication with project supervisor
Google Drive	To share work with supervisor and Engineers

Table 5.1. Project Tools

6 Project Plan

6.1 Sprint Schedule

The project spanned approximately 12 weeks which was organised into six two-week sprints. The first sprint focused on the RAMP deliverables, sprints 2-5 focusing on the actual development of the solution and finishing with the final sprint which was on dissemination as can be seen in Figure 6.1 below.

Sprint	Weeks	Tasks	Phase
Sprint 1	Weeks 1-2	RAMP completion: research, analysis, design, methodology, planning	RAMP
Sprint 2	Weeks 3-4	Database setup, synthetic data generation, initial Power BI use	Development
Sprint 3	Weeks 5-6	Fleet overview dashboard, pass/fail logic	Development
Sprint 4	Weeks 7-8	Tool level detail, slot-level detail display	Development
Sprint 5	Weeks 9-10	Additional time to bug fix, over a couple enhancements, finalise report	Development
Sprint 6	Weeks 11-12	Additional time to bug fix, over a couple enhancements, finalise report	Dissemination

Figure 6.1. Project Sprint Schedule

7 Implementation

This Section documents the technical implementation of the APAVS project. The project was completed in phases and from a Minimum Viable Product onto enhanced features.

7.1 Overview of Implementation

The implementation of this project worked in phases, due to the unfamiliarity with all the technologies it was decided to first try get a handle on Power BI. To get a core visualization CSV data was used as this is how the data is originally given. This allowed for more rapid prototyping and see what works before investing in a more well-defined database infrastructure.

7.2 MVP Dashboard

Development Environment

For the development of the MVP dashboard, we needed the fundamental building blocks for the creation of all the visuals. A list of the tools, load ports, and qualification runs would be used. To gather all of this information the original output of the tool would need to be examined. The original layout of the export file from the tool can be seen below in Figure 7.1.



MappingData.log	✓	05/02/2026 14:28	Text Document	203 KB
PodData1.txt	✓	05/02/2026 14:27	Text Document	531 KB
PodData2.txt	✓	05/02/2026 14:27	Text Document	531 KB
PodData3.txt	✓	05/02/2026 14:27	Text Document	531 KB
robot 1 config.txt	✓	29/01/2026 23:26	Text Document	2 KB
RobotCalibration.log	✓	05/02/2026 14:27	Text Document	4 KB
RobotConfig.log	✓	05/02/2026 14:28	Text Document	1 KB

Figure 7.1. Original File Structure

These files have all the necessary bits of information to make the visuals, but the folder is a bit verbose and convoluted. To have an easier time working with the data it was necessary to make 3 new files:

Tools.csv – The list of tools and their credentials for sign in. (relevant for pulling data.)

QualificationRuns.csv – Run Time, Taught positions, and Load port of run.

LoadPorts.csv – Port name and tool ID.

These files were then placed in a folder named “APAVS_Data” with the Power BI file as can be seen below in Figure 7.2.

 LoadPorts.csv		06/02/2026 16:20	Comma Separate...	2 KB
 QualificationRuns.csv		06/02/2026 16:20	Comma Separate...	4 KB
 Tool.pbix		06/02/2026 23:45	Microsoft Power B...	65 KB
 Tools.csv		06/02/2026 23:28	Comma Separate...	1 KB

Figure 7.2. Revised File Structure

This file structure maintained a clear separation between the data files and allowed for a future migration to a database in the next phases of the project.

Source Data Analysis

Before the files could be populated, an analysis of the existing data was conducted. This involved examining the original file structure as previous mentioned.

MappingData.Log Structure:

The mapping data contains the Z-position measurements for each of the slots. Each row entry would represent a single run. The layout is explained below in Table 7.1.

Field	Description	Example
Timestamp	Date and time of qualification run	25/01/2026 06:46
Port	Load port identifier (P1, P2, P3)	P1
Slots 1-25	Z-position measurements in millimetres	22.75, 23.89, ...

Table 7.1. MappingData.log Field and Descriptions

RobotConfig.log Structure:

The robot configuration file contains the taught Z-position values and their load port.

With the two above files above, we can then measure the qualification offset for each slot using the calculation: $Offset_n = Measured_n - (TaughtZ + 10 \times (n - 1))$

Where:

- $Offset_n$ = The offset for the slot n in millimetres
- $Measured_n$ = The Z-position for the slot n
- TaughtZ = The taught reference position
- n = Slot number (1-25)

We're following that the is qualification passing if it's absolute offset is within ± 1 mm of zero.

Data modelling design

The data model for the MVP was designed to support the structure required for fleet-wide analysis. This structure is compatible with the CSV files but is subject to some change in the future with the SQL server implementation. The Entity-Relationship Diagram can be seen below in Figure 7.3 which features each entity and its attributes.

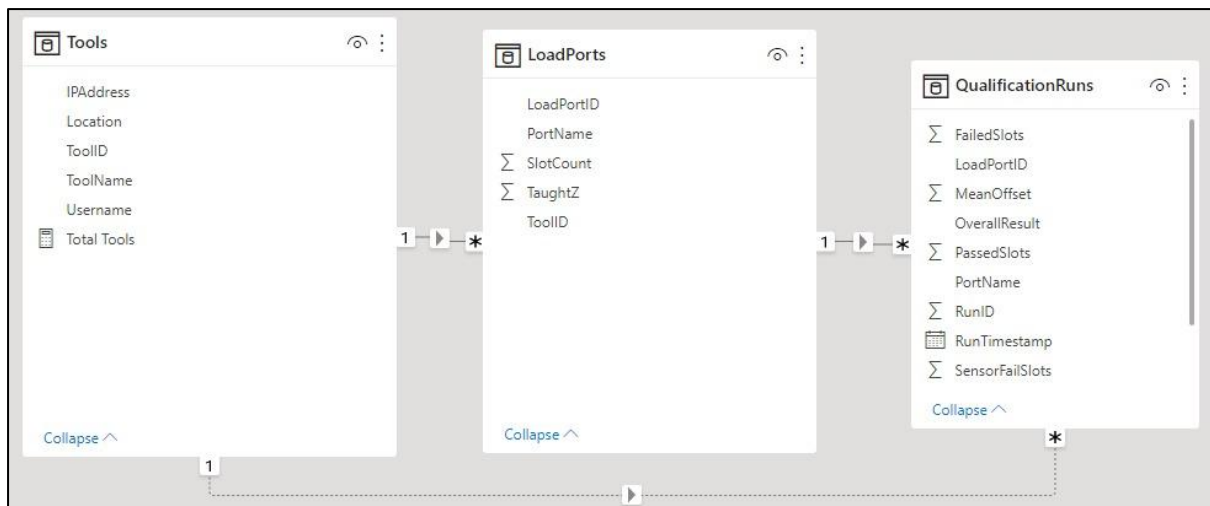


Figure 7.3. Entity-Relationship Diagram for MVP

The relationships between the above tables follow a one-to-many pattern:

- Tools (1) → LoadPorts (M): Each tool has many load ports.
- LoadPorts (1) → QualificationRuns (M): Each load port has many runs.

Data Generation

To test the setup on Power BI synthetic data was generated to represent values realistic to an actual run on the CSB Fleet. This was necessary to adhere to the SETU ethical guidelines. The data generated was designed to include scenarios which were discovered when analysing some of the authentically generated data. The scenarios included the following:

- Tools with consisting passing results (The majority of tool data reviewed)
- Tools with failing load ports
- Runs with sensors which had incomplete data set due to fail in sensors

Measurements (Dax Measures)

To create some of the graphs for the dashboard some custom measurements using the Data Analysis Expressions used in Power BI platform. Here are some of the custom metrics:

Measure 1: Total Tools

$$\text{Total Tools} = \text{DISTINCTCOUNT}(\text{Tools}[\text{ToolID}])$$

Measure 2: Pass Rate

$$\text{Pass Rate} = \text{DIVIDE}(\text{COUNTROWS}(\text{FILTER}(\text{QualificationRuns}, \text{QualificationRuns}[\text{OverallResult}] = "PASS")), \text{COUNTROWS}(\text{FILTER}(\text{QualificationRuns}, \text{QualificationRuns}[\text{OverallResult}] \text{ IN } \{"PASS", "FAIL"\})), 0)$$

Dashboard Visuals

The MVP dashboard design was centred around the user requirements found in Table 3.3 found on page 9. This prioritised quick visibility at a glance for the users to see the status of the fleet. Figure 7.4 shows the first prototype of the dashboard, we will quickly go through its functionality and capabilities. In the top third of the graph, you can see three KPI cards which display the total number of tools, number of runs, and the pass rate of the runs which will dynamically change depending on which of the filters (ToolID and Result) are selected in the middle third of the graph. On the right-hand side of Figure 7.4 you can also see each of the individual run IDs and their time stamp. On the bottom third of the graph, you have the run ID and result which are colour coded and show the complete number of runs per tool. Incomplete runs were excluded from both the numerator and denominator to avoid misrepresenting fleet health.

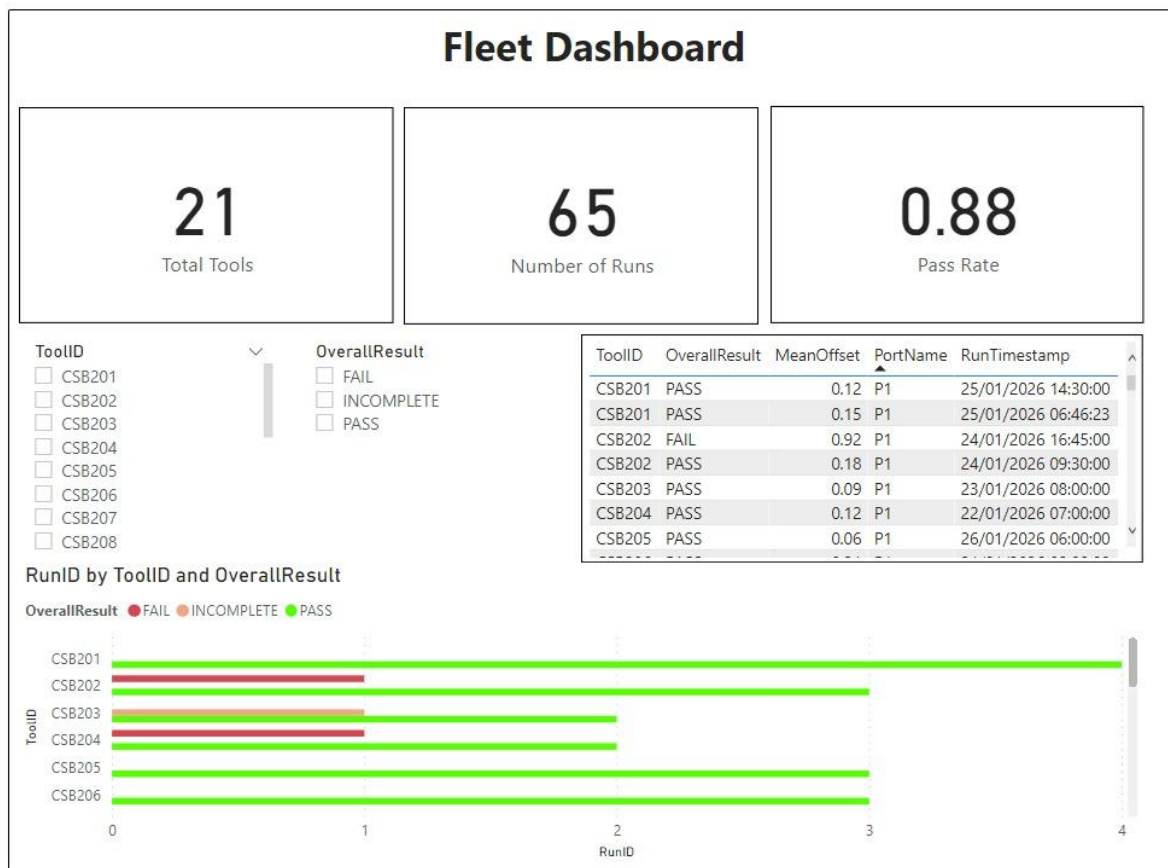


Figure 7.4. MVP Dashboard Layout

With this the MVP successfully meets the visibility requirements set out in Section “3.4 Feature Analysis” and allows for enhancement in further iterations. The three slicers on the fleet dashboard were later refined to make room for tool-specific measurements.

7.3 Phase 2: Database Migration and Enhanced Visualisation

In the second phase the data source was moved from CSV files to SQL Server Express, added additional formatting, and introduced additional KPI cards.

SQL Server Migration

SQL Server Express was installed locally (there was not yet a standard database to store the database) and the database APAVS_DB was created using SQL Server Management Studio (SSMS). The schemas for the database comprised of four tables: Tools, LoadPorts, QualificationRuns, and SlotMeasurements. A primary key was used for the LoadPorts combining the ToolID and PortName, as port names (LP1, LP2, LP3) are only unique within the

given tool. The CEID column was also an addition given that the tools belong to different tool groups (CSBca and CSBcd) which have different port configurations, CSBca tools having two load ports with CSBcd having three.

Power BI was then reconnected using the native SQL Server connector. A benefit of this was that there were no additional drivers required to achieve the connection, and the relationship tables were automatically detected.

Conditional Formatting

Colour based pass/fail indicators were included to the results table. Power BI's rules-based formatting only allows for numeric values so a helper measure was created to help map the text results to hex based colour codes. The PASS results were mapped to green (#107C10) and FAIL to red (#D13438). Conditional formatting can be seen below in Figure 7.5.

ToolID	CEID	PortName	OverallResult	RunDateTime
CSB222	CSBcd	LP1	INCOMPLETE	17/10/2025 00:57:14
CSB222	CSBcd	LP1	INCOMPLETE	24/10/2025 03:02:52
CSB222	CSBcd	LP1	INCOMPLETE	28/10/2025 06:41:24
CSB222	CSBcd	LP1	PASS	22/09/2025 06:02:11
CSB222	CSBcd	LP1	PASS	22/09/2025 07:19:15
CSB222	CSBcd	LP1	PASS	22/09/2025 11:47:39
Total				

Figure 7.5. Conditional Formatting Included in the Overall Result Column

Additional KPI Cards

A Last Updated card was added by displaying the timestamp of the most recent qualification run, formatted as DD/MM/YYYY HH:MM. This addresses the user story US-03 in which the technicians need to know if the data they are looking at is current.

7.4 Phase 3: Slot-Level Measurement and Trend Analysis

For the third phase a SlotMeasurements table was included to enable the insight of the foup to a specific slot data and enable the historical data for a period of time.

SlotMeasurements Table

A new table was created in SQL to store the 25 individual Z-position measurements for each of the qualification runs. Each row contained the RunID, SlotNumber, and the measured Z value. This expanded the Entity Relationship diagram from 3 to 4 as can be seen in Figure 7.6.

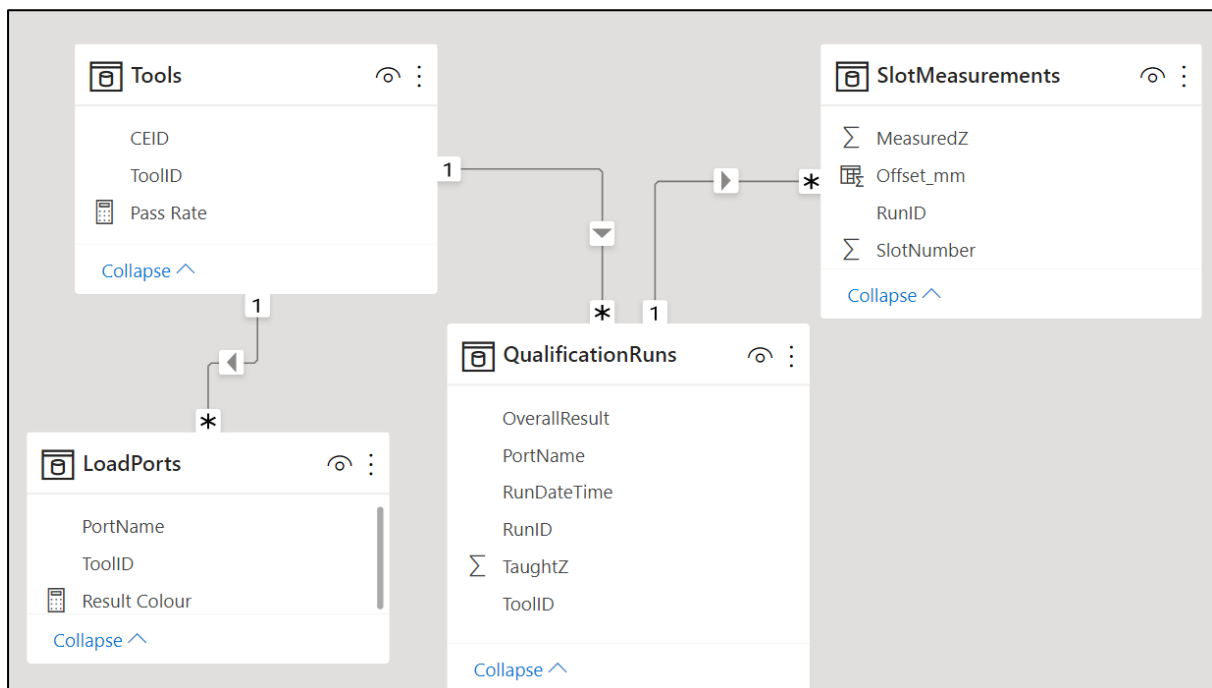


Figure 7.6. Updated ER Diagram

To complete the offset calculation of the slot measurements a DAX calculated column was added to the SlotMeasurements table. Below was the calculation to be completed in the table:

$$\begin{aligned} \text{Offset_mm} &= \text{VAR TaughtZ} \\ &= \text{LOOKUPVALUE}(\text{QR}[\text{TaughtZ}], \text{QR}[\text{RunID}], \text{SM}[\text{RunID}]) \text{ RETURN ROUND}(\text{SM}[\text{MeasuredZ}] \\ &\quad - (\text{TaughtZ} + (10 * (\text{SlotMeasurements}[\text{SlotNumber}] - 1))), 2) \end{aligned}$$

Where:

- *QR* = QualificationRuns
- *SM* = SlotMeasurements

Trend Chart

A line chart was then added to a new report page with the RunDateTime on the X-Axis and the Average Offset_mm on the Y-axis. Four constant reference lines were added to the line chart at the OOC limits $\pm 1.0\text{mm}$ and the warning thresholds $\pm 0.4\text{mm}$. This also links back to the US-05 story – monitoring a drift because a load port is to go out of control. An example of the trend chart can be seen below in Figure 7.7 where we can see the offset values within the OOC limits.

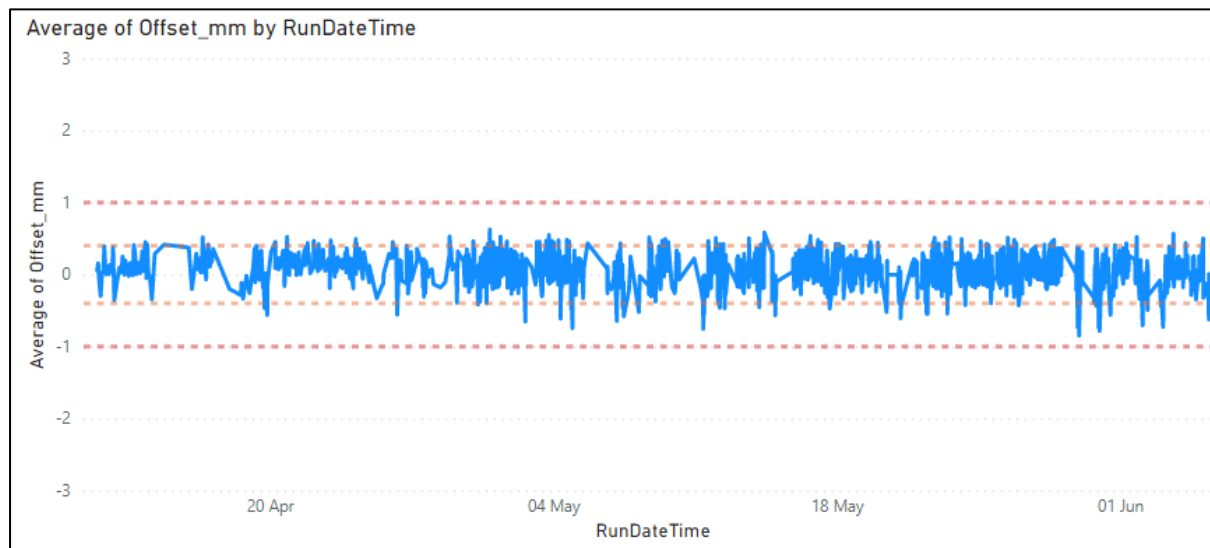


Figure 7.7. Trend Chart of a Tool

7.5 Phase 4: Automated Extraction Pipeline

The most complex phase of the project was to replace the manual data workflow with an automated PowerShell pipeline. This phase involved going into the existing VBA macro and “reverse-engineering” what had previously existed. Other steps also included building network connectivity to the CSB tool fleet, parsing the raw sensor data, and transferring the data from a remote desktop environment back to the local SQL database. iGPT, an in-house AI model running on Claude Sonnet 4 (Anthropic, 2025), was utilized *heavily* during phase of the project as it was crucial in helping to understand the existing code and create scripts to replace it.

Breaking Down the Existing Workflow

The starting point was the existing FI Mapping Macro, essentially an excel file containing VBA code and inline PowerShell scripts which allowed engineers to manually analyse individual tools. Upon analysing the VBA code, the data flow could be reviewed:

1. Authenticated the connection via the net use command
2. Retrieved the most recent FI Log zip files depending on their origin
3. Extracted the PodData and robot configuration files from the zip
4. Parsed the above files into the MappingData.log and RobotConfig.log formats

From the existing steps an automated pipeline was then formulated.

Configuration

A JSON configuration file was created to hold on to the tools names and their associated IP addressed which allowed for network connection. This would also allow for removing or including future tools to the configuration file without large modifications taking place. The real configuration files were excluded from the GitHub repository as it is sensitive data and a placeholder “Apavs Config.example” was added for review instead.

Network Architecture

This is where a widespread database assessable from each workstation complicates things. Each of the tools are only accessible when use a remote desktop session on the Intel network, I’ll refer to this as the “Tool Network”. The SQL Server was installed the author’s personal laptop which could not connect to the tools directly. Due to this a two-stage extraction/Import was required. First APAVS_Extract.ps1 ran on the Tool Network to retrieve and parse tool data and exporting the results to CSV. These CSVs were then transferred to the laptop via a Windows networks share (SMB), where APAVS_Import.ps1 could then be used to read them on the SQL Server. In future iterations of the project the database could exist on the Tool Network but there was not yet a reasonable way of implementing this. An illustration of the pipeline as it exists can be found on page 23 in Figure 7.8.

Tool Connection

Each of the tools were accessed using a ping test first to verify connectivity and then followed by using authentication with credentials prompted securely at runtime via Get-Credential rather than it being hardcoded. During testing all tools but CSB205 was reached which is accurate because it had been previously decommissioned.

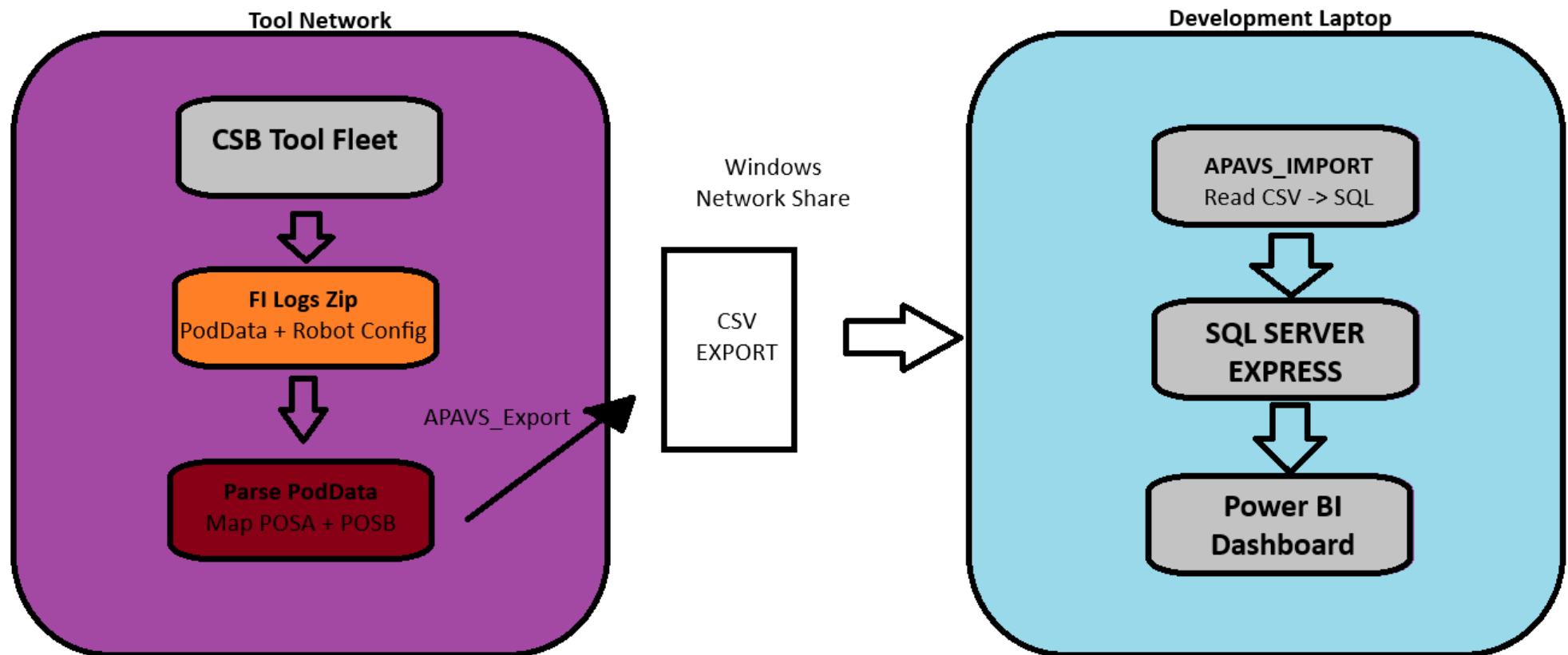


Figure 7.8. APAVS Data Pipeline from Tool Network to Development Laptop

Parsing the Raw Data

The raw PodData files contained FOUP placement events, mapping results, and other diagnostic information. The parser filtered for both MAP-POSA/B line pairs. MAP-POSA contained the Z-position measurements for slots 1-12 while MAP-POSB contained the Z-position measurements for slots 13-25. These were then compiled into 25 rows were a timestamp and their applicable load port identifier. PodData1.txt corresponded to LP1, PodData2.txt to LP2, and LP3 the same. A date filter “DaysBack” could be used to limit extract to a specific window of time, which is useful if only looking for data after a specific maintenance event.

Offset Calculation and Result Classification

In previous iterations the offset formula was applied identically as implemented in previously:

$$\langle\langle Offset \rangle\rangle_n = \langle\langle Measured \rangle\rangle_n - (TaughtZ + 10 \times (n - 1))$$

But a significant design decision had to be made during testing after reviewing some of the results. On the tool CSB201 had previously shown 167 failures but upon reviewing the results it revealed that the vast majority had zero failed slots. These failures were caused by a sensor reading of 0.00 which indicated that a sensor did not read rather than the reading being a failure. So, three tiers of classification were implemented in the testing:

PASS: All slots within $\pm 1\text{mm}$ reading, not including 0.00 values (sensor values).

FAIL: One or more slots with an absolute offset of more than 1mm.

INCOMPLETE: One or more slots returned with a value of 0.00.

After testing the previous tool, with this new logic it was discovered that of 167 fails only 4 were genuine failures as seen in Figure 7.9 below.

```
OLD logic (no INCOMPLETE category):
PASS: 323
FAIL: 167
Total: 490
NEW logic (with INCOMPLETE category):
PASS:      323
FAIL:       4
INCOMPLETE: 163
Total: 490
```

Figure 7.9. Old and New Logic with and Without Incomplete Category

Temporal Z-Value Mismatch

Testing on CSB231 revealed that of 738 runs 721 were fails. But upon inspection of the graph, it appeared as though the values at with a $\pm 1\text{mm}$ reading and these runs were not previously flagging. This is a known limitation of the previous system; the robot configuration files only contain the TaughtZ value from the most recent reteach event. This means that although these values would have been passing previously the classification is now FAIL because it is comparing it to the most recent TaughtZ value. This reinforces the original design decision to store each of the TaughtZ values with each qualification run in the database which allows the reference values at the time of the measurement. In Figure 7.10 below we can see the previous offset values between a $\pm 1\text{mm}$ measurement followed by 2 different maintenance events circled in red.

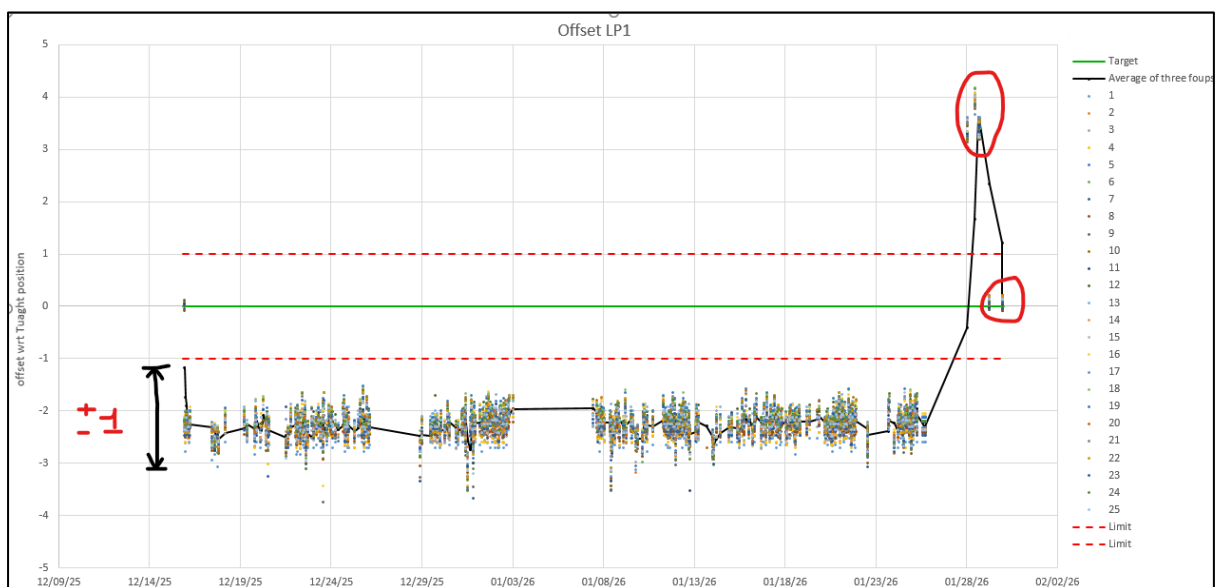


Figure 7.10. Mismatch of TaughtZ Value Over Time

Fleet Extraction and Results

The full extraction and import pipeline were executed across all 11 CSB tools in the configuration. The complete extraction process takes some time to complete as it has to first ping the tool, connect, locate the zip folder, parse all the runs for each of the load ports, calculate the offsets for each of the runs and finally export and copy to the development laptop. This complete process can be caught in the PowerShell window for each of the tools as can be seen in Figure 7.11 on page 26 .

```

=== Processing CSB231 (CSBcd) ===
Pinging CSB231 (10.44.49.225)... Reachable
Connecting to CSB231... Connected
Found zip: ficlogFiles_1-29-11_26_48.zip (57 days old)
Robot config copied from: robot 1 config.txt
Zip format: Legacy (ficlogs.tmp)
Parsed 338 runs for P1 (last 365 days)
Parsed 352 runs for P2 (last 365 days)
Parsed 412 runs for P3 (last 365 days)
P1 TaughtZ = 25.34
P2 TaughtZ = 25.15
P3 TaughtZ = 25.23
Offsets: 1102 runs (14 PASS, 1085 FAIL, 3 INCOMPLETE)
Exported and copied to laptop
=== CSB231 complete ===

```

Figure 7.11. Process for a Single Tool

Once the complete extraction is complete the command gives a tally of all the tools successfully processed and the final elapse time as seen in Figure 7.12 below.

```

=====
Extraction Complete
Tools processed: 11
Tools failed:    0
Time elapsed:    9.3 minutes
=====

```

Figure 7.12. Extraction Complete Message

8 Future Work

In the following Sections I will outline some of the enhancements that would have improved the system if additional development time was made available.

8.1 Automated Scheduling

The current pipeline isn't completely automated and still requires manual execution from both the Tool Network (using APAVS_Extract.ps1) and on the development laptop (using APAVS_Import.ps1). An obvious extension of this would have a scheduled extraction/import at regular intervals such at the beginning of each shift or every day. This would allow the APAVS dashboard to be regularly updated without requiring manual intervention.

8.2 Global Database Environment

The current architecture, as presented previously in Figure 7.8 on page 23, requires a two stage process due to the Tool Network and development laptop being on separate network segments. Deploying an SQL Server on a machine accessible from both the Tool Network and user workstations would eliminate the need for this additional step and would allow a direct import of data from the tools. This was not feasible as a global database doesn't currently exist and one could not be created on the Tool Network for the sake of this project.

8.3 Historical TaughtZ Position

The temporal mismatch, as identified in Section 7.5, identified that the historical values of the offset position are evaluated against the current TaughtZ value rather than the value of the TaughtZ at the time of the measurement. An enhancement in the future would be matching historical values with the TaughtZ values at the time of the measurement. This would resolve false failures as seen in Figure 7.10 on page 25.

8.4 Drift Alerting

With sufficient historical data stored in the SQL database, the system could implement alerting when the load port's mean offset approaches beyond the $\pm 0.4\text{mm}$ warning threshold. In workplace practice this would normally be linked to E3A alert system but could have also been achieved through Power BI data-driven alerts or other monitoring scripts that could send notifications via an email. This would have been identified in the user story US-05 but was beyond what was achievable with the current implementation of the project.

8.5 Expansion to Additional Tool Sets

When reviewing the additional tool sets on either side of the factory both F24 and F34 the way in which the load port mapping was achieved ranged from similar to completely different. If it was as simple as adding additional tool sets to the JSON configuration file and doing slight modifications to the parsing functions this would have been achievable but how the tools and files are configured it was found that it would be vastly more complex a task. With some time

and additional help, the addition of these tools' sets would be possible in most cases with the core functionality of a connect being made, retrieve of files , parsing, calculations and store.

8.6 Overview

When referring back to Section 3.4 Feature Analysis it can be noted that all five of the MVP features identified were delivered. Of the four advanced features, historical trending and exporting capabilities were fully implemented. Tool navigation was mostly delivered through the slicer-based filtering and drift alerting was noted as future work.

Feature	User Stories	Description	Status
Fleet Overview	US-04,US-07	Page 1 dashboard shows all tools with pass/fail/incomplete status, bar chart by tool	MET
Tool Detail View	US-01,US-02	Page 2 trend chart with tool and port slicers, slot-level data in SlotMeasurements table	MET
Pass/Fail Calculation	US-01,US-07	Offset formula implemented in both DAX and PowerShell, extended to three-tier classification	MET
Time Stamp	US-03	Last Updated KPI card showing most recent qualification run	MET
Filtering	US-06	ToolID, PortName, and OverallResult slicers.	MET
Historical Trending	US-05	Trend chart on Page 2 with $\pm 1\text{mm}$ and $\pm 0.4\text{mm}$ reference lines	MET
Drift Alerting	US-05	Identified as future work in Section 8.4. The data and thresholds exist but automated notifications not yet implemented.	Out of scope
Tool Detail Navigation	US-08	Slicers allow selecting specific tools and ports, but no dedicated per-tool page with advanced navigation	Partially met
Export Capability	N/A	CSV export built into the extraction pipeline, results and slot-level data exported per tool	MET

Table 8.1. User Stories and Status

9 Critical Self Review

In this Section a review of what has been learned undertaking this project, what has been achieved, and problems which have been identified in the process.

9.1 What I have learned

Power BI and DAX

Prior to the commencement of this project Power BI had not been used in any capacity. With this project the author has learned how to build visuals, configure relationships between the tables and their individual entities, and write DAX expressions. One of the hurdles which was difficult to overcome at first due to the level of experience was implementing conditional formatting. The rules-based formatting only accepted numeric values, so a work around was required using helper measures to map out the text results to hex colour codes. Another issue which was encountered was the Related function failing to calculate the offset values due to the cross-filtering direction. Switching to LookUpValue which explicitly matches the RunID resolved this issue without having to reevaluate the relationship direction.

SQL Server Express

SQL Server Express was another learning opportunity for the project as it's ability to connect to the Power BI without requiring third-party applications and its free user tier made it a perfect fit for the project. A practical lesson which was learned early on in the project was failed connections due to self-signed certifications and having to configure settings to allow for this option. Having previously used MySQL in previous projects was a help although the two applications are similar but differ in their query syntax and procedural structures. One thing that was also required was a composite primary key as in the LoadPorts table no single column alone was unique, so a combination of both the ToolID and PortName was used for a unique identifier.

PowerShell

Although PowerShell was used previously the code required for the project wouldn't have been possible without the assistance of generative AI. The extraction script was tested incrementally throughout phase 4, and each part of the script was tested individually in the

terminal. As an example of this testing the PodData parsing logic was first tested by splitting a single line and then extracting the Z values for one MAP-POSA/B pair and then looping through the entire file. This incremental approach helped build an understanding of how PowerShell handled arrays, regex matching, and object creation. One lesson was understanding the difference between normally executing a script and dot-sourcing it, as it was learned that dot-sourcing preserves functions in the current session.

Reverse-Engineering Existing Code

Analysing the existing VBA code which was contained in the FI Mapping Macro was one of the more educational parts of the project. The macro contained Shell commands, inline PowerShell scripts embedded in the code as string variables, and Windows networking commands. Reading through this code an understanding was eventually made of what had previously been a sort of black box. The scripts which were later produced borrowed much of how tools were authenticated using net use with admin shares, how the zip files were located and extracted, and how the raw PodData was parsed into the clean MappingData format. Without the analysis of this existing code the pipeline would have not been possible.

Domain Knowledge

The knowledge of technicians and engineers was also required to complete the system as technical skills alone wouldn't have made it possible. An understanding of why the measured Z-values of 0.00 represents a sensor failure rather than a failure may have been deduced but having a confirmation of the fact puts more confidence in the system. Also, understanding that the TaughtZ values change after reteaching events and that the config file only reflects the current state required experience with the maintenance workflow. These insights came from working on the tool as a technician and reviewing these systems previously with engineers could not have been learned from documentation alone.

9.2 What I Achieved

This project delivered a system that addresses a genuine operation gap at the Leixlip facility. Although there is room for enhancements to be made in the future the following are some of the key achievements on the system so far.

Feature Delivery

All five of the MVP features identified in Section 3.4 Feature Analysis were completed. Fleet overview showing all the tools with pass/fail status calculation, time stamps on the KPI cards showing the most recent runs, and many KPI cards for filtering which allow deep dives into the specifics of a tool and it's runs as can be seen below in Figure 9.1.

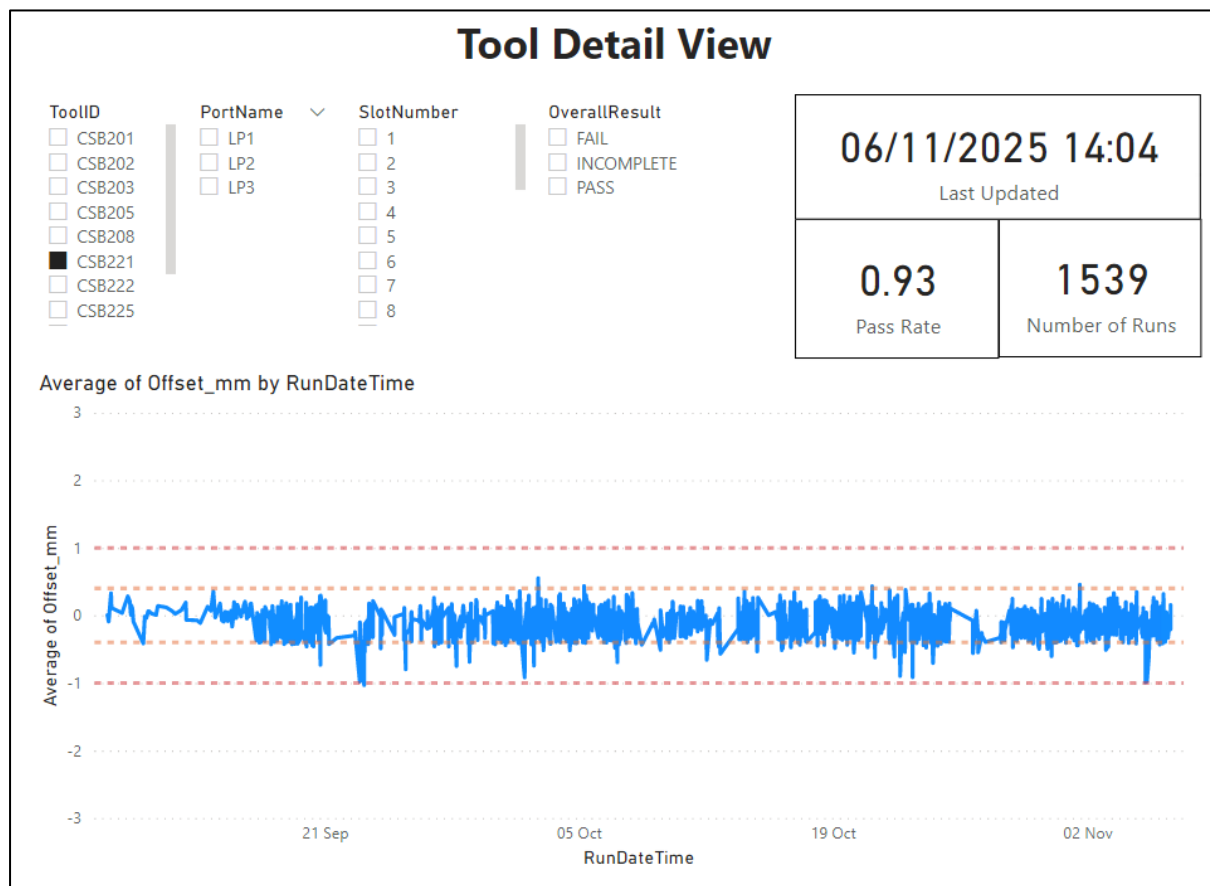


Figure 9.1. Tool Detail View

Of the four advanced features, historical trending and export capability were fully implemented, Tool detail navigation was mostly met but lacks a functional drill through mechanic linking both the first and the second pages together. Drift alerting was documented as future work in Section 8.4.

Automated Pipeline

The PowerShell pipeline successfully connected to 10/11 tools in the CSB fleet, which is accurate due to CSB205 being decommissioned so acted as a good control. The full extraction of the data takes approximately 9 minutes, replacing a manual process that would have required the technician or engineer to connect to all the tools individually with the excel macro and then visually inspect each and every result. What is now a two-step process can be further reduced to a one step process once a centralised database on the Tool Network has been established.

Data Processed

The pipeline was able to process 10,684 qualification runs with 270,100 slot run measurements imported into the SQL Server Database. All these runs were not previously accessible in one easy to reach consolidated form prior to the APAVS project. Each tools data only existed temporarily in an Excel sheet with no storage.

Three tier classification

During the completion of the project, it was found that the two-tier classification wouldn't be suitable for the Dashboard as it neglected to address the issue of sensor failures (MeasureZ values of 0.00). This hurdle was overcome by implementing the INCOMPLETE classification which prevented the production of misleading results. This newly introduced classification, which did not previously exist, provided a significantly more accurate portrayal health of the tool fleet. Tools which previously thought to have been in more health in many cases were predominantly affected by poor sensor reads rather than wafer positional failures.

Future Direction

As previously mentioned in Section 8 Future Work there were five areas required for future development: An automated scheduling to remove the need for manual execution, a centralised database that would eliminate for a two-step process, a better resolution for the historical TaughtZ position using the historical records for the time period the measurement was taken, drift alerting, and an expansion on the tool sets beyond the Fab24 CSB fleet. APAVS was deliberately designed with configuration in mind so expansion should be straightforward.

9.3 Problems Encountered

In this Section a conversation will be had about the problems encountered during the creation of the APAVS system and how these issues were overcome.

Network Separation (Tool Network & Development Laptop)

One of the largest technical challenges of the APAVS system was that data was only available for the Tool Network while the database could only exist on the development laptop. There was an attempt to work around this and to see if OneDrive was synced between both the networks (it wasn't) and testing if the Tool Network desktop could ping the development laptop (thankfully it could). A Windows network share was used by establishing a folder on the laptop and enabling file and printer sharing through the firewall. This allowed the extraction of the CSV files to be directly transferred to the laptop where the import script could read the files onto the SQL Server. While not as seamless as was previously hoped this two-stage approach allowed the entirety of the pipeline to be demonstrated.

A sub issue relating to this was all the corporate network restrictions when working with the project. Trying to connect with GitHub was impossible through using a browser and was only found to work when using the GitHub Desktop instead. Trying to transfer information from the development laptop to my personal computer such as screen shots and code snippets was tedious due to different firewalls and other blocks that were in place to stop the transferring of sensitive data. These small little inconveniences and many more compounded into "wastes of time" which could have been more productive but also reflected some of the frustrations of a real-life environments where these occurrences regularly take place.

Sensor Fail Classification

Testing of the extraction pipeline early on in the project show that on CSB201 that 335 of the 998 runs were reported as a failure. This was speculated to be far too high for a tool that was considered healthy and wasn't suffering from any tool related error due to wafer misplacement which would likely occur if the wafers were misaligned. Examining the results more closely revealed that the majority of the results that were classified as failures were flagged because the sensors returned a MeasuredZ value of 0.00. The value of 0.00 indicates

that the wafer position wasn't read rather than it being in a failing position. This was not handled because by the existing Excel Macro meaning that the problem was either unnoticed or more than likely not addressed. The solution was then to implement the three-tier classification that now exists (Pass, Fail, Incomplete) which correctly separated genuinely failing results from the sensor read issues.

Stakeholder Availability

A significant non-technical challenge was staying in direct communication with the workplace mentor Kevin O'Dea during the course of the project. As Head Engineer on the CSB tool set his feedback on design decisions and domain knowledge was crucial to the completion of the project. However, for 8 of the 12 development weeks the author was assigned to work the night shift pattern while Kevin remains on day shift. This would leave small windows at the end of my shift (start of his) when other agendas such as passdown and team meeting occurring most frequently. This was further exacerbated when for 2 of the 4 weeks there was an overlap of our shifts, he was out due to medical reasons. This left a very limited window of opportunity for feedback in-person.

This was also compounded by a failure on my end to establish a regular meeting schedule or alternative means of communication early on in the project. Had the author had made a stronger effort to establish for periods of feedback through other means such as email or teams this availability could have been reduced. This is a lesson in stakeholder management which will need to be addressed in future projects.

Temporal TaughtZ Mismatch

Testing on CSB231 showed that of 738 runs 721 were classified as a Fail which upon reviewing the data was known to be incorrect. The trend chart showed offset values sitting within a value of $\pm 1\text{mm}$ but shifted on the Y-axis. When looking at the robot configuration file it was noted that the only the most recent reteach value was stored so that the offset values were being calculated from only the most recent reteach. This means that the values were in reality passing at the time the values were taken but now when being compared to the new TaughtZ values they were being classified as Fail as can be seen in Figure 7.10 on page 25. This was

documented as a known limitation and was identified as future work in Section 8.3. This also reinforced the decision to store each TaughtZ value in the database rather than relying on the single value given with the Robot Calibration file.

9.4 Use of Generative AI

Generative AI tools were used throughout the project for debugging and code explanation at the beginning of the project but from the beginning of Phase 4 of the project the use of an internal Intel deployment of Claude (iGPT) was used more readily. In particular AI assistance was used in the following areas: PowerShell Script development and syntax, SQL query construction, assisting with DAX formula creation.

In parts where AI Generation was particularly heavy is where the complexity of the PowerShell pipeline exceeded the author's experience. Each function was built and incrementally tested against live tool data to verify that the code was correct, and the approach was to understand the code rather than just accept what was being generated.

The following aspects of the report had no support from AI and relied on the author alone: identification of the operational issue as an experienced technician, reverse-engineering the VBA macro to understand the existing code, design decisions to implement an Incomplete classification based on domain knowledge of this sensor issue, identification of the temporal mismatch, and the network architecture decisions as a response to the firewall and other restrictions which exist on Intel's platforms.

As much as it could AI was used as an aid where there was the lack of a stakeholder or for the replacement for experience coding in this environment. The code in this report was reviewed and rewritten to gain an understanding.

10 References

- Bridge Tronic Global. (2025).** *Applied Materials Endura 2 PVD (Physical Vapor Deposition)*. [online] Available at: <https://www.bridgetronic.com/products/79749> [Accessed 15 Jan. 2026].
- IBM. (2024).** *IBM IndustryView for Semi-Conductor Standard - Overview*. [online] Available at: <https://www.ibm.com/products/siview> [Accessed 15 Jan. 2026].
- Critical Manufacturing. (2026).** *Manufacturing Execution System (MES)*. [online] Available at: <https://www.criticalmanufacturing.com/industries/semiconductor-manufacturing/> [Accessed 15 Jan. 2026].
- KLA. (2023).** *Semiconductor Software Solutions / KLA*. [online] Available at: <https://www.kla.com/products/software-solutions/semiconductor>. [Accessed 15 Jan. 2026].
- yieldWerx. (2025).** *yieldWerx – Zero-Defect Future Predictive AI Yield Analytics*. [online] Available at: <https://yieldwerx.com/> [Accessed 15 Jan. 2026].
- Hosseinglm (2025).** *GitHub - Hosseinglm/Semiconductor-Insights: Data-Driven Manufacturing Analysis Dashboard*. [online] GitHub. Available at: <https://github.com/Hosseinglm/Semiconductor-Insights> [Accessed 15 Jan. 2026].
- shirishpandagare (2025).** *GitHub - shirishpandagare/SECOM_Project: Final project done in Engineering Data Analysis (ISEN 613)*. [online] GitHub. Available at: https://github.com/shirishpandagare/SECOM_Project/ [Accessed 15 Jan. 2026].
- Cohn, M. (2019).** *User Stories and User Story Examples by Mike Cohn*. [online] Mountain Goat Software. Available at: <https://www.mountangoatsoftware.com/agile/user-stories>. [Accessed 22 Jan. 2026].
- Anthropic. (2025).** *Claude (Sonnet 4 version) [Large language model]*. Deployed internally as iGPT at Intel Corporation. [Accessed 30 Mar. 2026]